

FT3169 驱动移植指南

FT3169 驱动移植指南	
Project name	FT3169
Document ref	[Document ref]
Version	1.0
Release date	2022.06.15
Owner	欧阳云凤
Classification	
Distribution List	
Approval	

This document contains information proprietary to FocalTech Systems, Ltd., and may not be reproduced, disclosed or used in whole or part without the express written permission of FocalTech Systems, Ltd.

Copyright ©2022, FocalTech Systems, Ltd

All rights reserved

22th Floor,Block B3, Building 9,
Zone II of Shenzhen Bay Eco-Technology Park,
Yuehai Avenue, Nanshan District, Shenzhen, Guangdong Province, China

ZIP :518057

T +86 755 26588222

F +86 755 26712499

E support@focaltech-systems.com

www.focaltech-systems.com

Revision History

Date	Version	List of changes	Author	Approved by
2022.06.15	1.0	1. Initial draft.	OuYangyunfeng	Luoguojin

Table Of Content

1 概述.....	1
2 基础接口设置.....	2
2.1 I2C 通讯接口设置.....	2
2.2 延时函数	2
3 功能模块移植.....	3
3.1 初始化.....	3
3.1.1 初始化参数	3
3.1.2 TP 初始化	3
3.2 触摸功能	4
3.2.1 多点触摸.....	4
3.2.2 手势模式.....	4
3.3 亮灭屏机制.....	4
3.4 ESD 检测机制	5
3.5 固件开机升级	5
3.5.1 固件开机升级流程图.....	5
3.5.2 固件开机升级移植	6

1 概述

本指南介绍如何移植“FocalTech FT3169 Driver Code”，包括初始化、多点触摸、手势模式、ESD 检测机制和固件升级等功能。

注意：

- 1) 该文档涉及的 I2C 通讯接口、GPIO 配置使用、定时器使用以及 Delay 函数，请参考你的平台做相应适配；
- 2) 如下功能为**必须移植的功能**：
 - ◆ 触摸报点，具体实现请参考本文档[触摸功能](#)
 - ◆ ESD 检测机制，具体实现请参考本文档[ESD 检测机制](#)
 - ◆ 固件升级，具体实现请参考本文档[固件开机升级](#)
- 3) 如下功能为可选移植项：
 - ◆ 手势模式，具体实现请参考本文档[初始化参数](#)以及[手势模式](#)

2 基础接口设置

本章节介绍跟平台相关的基础配置, 需要根据平台做适配, 主要包括: I2C 通信接口和 delay 延时函数。

2.1 I2C 通讯接口设置

第一步, 根据当前平台在 focaltech_core.c 文件中初始化 I2C 接口, 请填写初始化接口函数

```
static int platform_i2c_init(void)
```

第二步, 更换 I2C 读写函数为当前平台 I2C 读写函数:

- `int platform_i2c_write(uint8_t addr, uint8_t *wbuf, uint16_t wlen):` addr 为写入的地址, data 为写入的数据 buffer, datalen 为写入的数据 buffer 的长度;
- `int platform_i2c_read(uint8_t addr, uint8_t *rbuf, uint16_t rlen):` addr 为读取的地址, data 为读取的数据 buffer, datalen 为读取的数据 buffer 的长度。

Sample code 中实际调用的 I2C 读写接口为: (会自动调用上面 platform_i2c_write/read 函数)

```
int fts_write(uint8_t addr, uint8_t *data, uint16_t datalen)  
int fts_read(uint8_t addr, uint8_t *data, uint16_t datalen)
```

请注意: I2C 读写函数需要在 I2C 传输失败时, retry 2 次, 总共 3 次通信来确保本次寄存器操作是否异常。

2.2 延时函数

延时函数 `void fts_msleep(unsigned long msec)` 涉及的功能为延时 msec 毫秒, 请根据平台配置该函数。

fts_msleep() 函数默认函数原型为:

```
void fts_msleep(unsigned long msec)  
{  
    usleep(msec * 1000);  
}
```

3 功能模块移植

本章节介绍所有具体功能模块的移植。

3.1 初始化

3.1.1 初始化参数

首先在 focaltech_core.c 中初始化结构体 `static struct fts_ts_data _fts_data:`

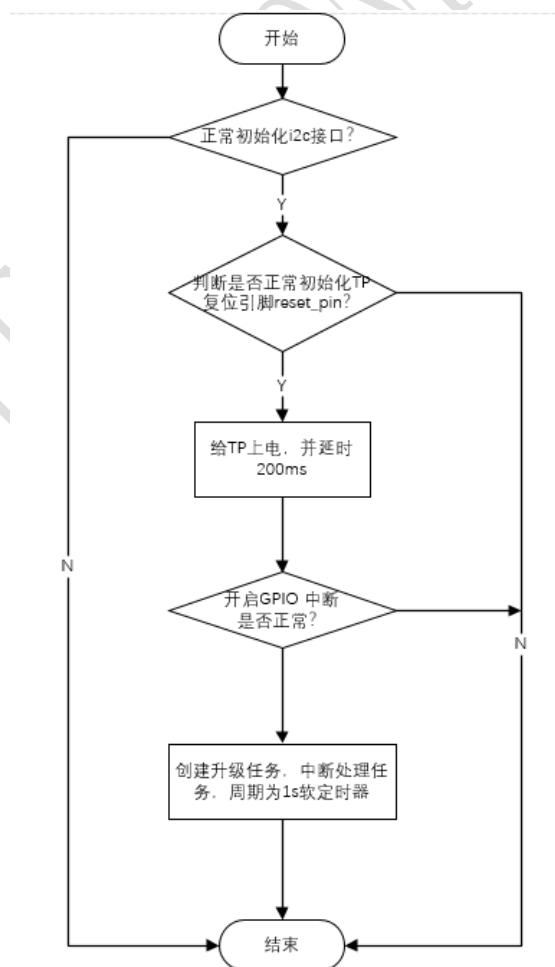
```
static struct fts_ts_data _fts_data = {  
    .suspended = 0,  
    .gesture_support = FTS_GESTURE_EN,  
    .esd_support = 1,  
};
```

具体参数定义：

- suspended: 0 为 normal mode (亮屏)，1 为 suspend mode (灭屏)；
- gesture_support: 0 为不开启手势模式，1 为启用手势模式，其中宏 `FTS_GESTURE_EN` 在 focaltech_core.h 中配置；
- esd_support: 0 不需要进行 esd 检测，1 开启 esd 检测，默认开启 esd 检测。（必须添加且默认使能）

3.1.2 TP 初始化

3.1.2.1 TP 初始化流程图



3.1.2.2 TP 初始化 sample code

TP 初始化的 sample code 是通过调用 focaltech_core.c 中 `int fts_ts_init(void)` 函数来实现的，与 TP 初始化流程图一一对应，具体如下：

- 1) `static int platform_i2c_init(void)`: I2C 初始化，根据当前平台初始化 I2C 通讯接口，包括：I2C pins 初始化、速率、I2C 地址等功能；
- 2) `static int platform_reset_pin_cfg(void)`: reset 引脚初始化，根据当前平台编写 reset pin 配置，配置为输出模式，并输出低电平（在上电之前，reset pin 需保持为 Low）；
- 3) `static void fts_power_on(void)`: 根据 TP 规格书上描述的 power-on sequence，给 TP 上电，并同时控制 reset pin，并且延时 200ms，等待 TP 设备初始化完成；
- 4) `static int fts_check_id(void)`: 读取 TP 设备 ID，检查当前的 TP 设备是否正确，若不正确直接退出；
- 5) `static int platform_interrupt_gpio_init(void)`: GPIO 中断初始化，根据当前平台配置 INT pin 为中断模式且上升沿触发模式，中断触发对应的中断向量入口函数为 `void FTS_INT_IRQHandler(void)`，用于获取报点数据；
- 6) `void fts_AppTaskCreate(void)`: 任务管理器，创建升级任务处理函数、报点数据处理任务处理函数、定时器周期为 1s 的定时器用于 ESD check 使用。

3.2 触摸功能

在 INT pin(GPIO)中断服务函数中处理多点触碰/手势模式，中断服务入口函数根据当前平台定义，sample code 中中断触发后发送队列消息，进入 `static void FTS_Int_Task(void* parameter)` 任务处理，处理函数为：

```
int fts_gpio_interrupt_handler(void);
```

3.2.1 多点触摸

若手势模式未开启（宏 `FTS_GESTURE_EN = 0`），则中断触发后获取触摸点数据，根据当前获取的触摸点数据可进行相应处理，sample code 中对应处理函数：

```
static int fts_touch_process(void);
```

3.2.2 手势模式

若手势模式开启（宏 `FTS_GESTURE_EN = 1`），则中断触发后并且处于 suspended 则进行手势处理，根据当前获取的手势可进行相应处理，sample code 中对应处理函数：

```
static int fts_gesture_process(void);
```

注意：

- ◆ 若 INT 脚中断产生时 TP 处于睡眠状态才能进行手势数据的读取；
- ◆ `int fts_ts_suspend(void)` 进入睡眠模式（灭屏），对应标志 `fts_data->suspended = 1`；
- ◆ `int fts_ts_resume(void)` 退出睡眠模式，进入正常模式（亮屏），对应标志 `fts_data->suspended = 0`。

3.3 亮灭屏机制

整机系统在亮灭屏时，需要调用对应函数设置 TP 的状态，具体为：

灭屏时，调用 `fts_ts_suspend()` 函数设置 TP 为 sleep 或者手势模式。

亮屏时，调用 `fts_ts_resume()` 函数，复位 TP 成 Normal 工作模式。

3.4 ESD 检测机制

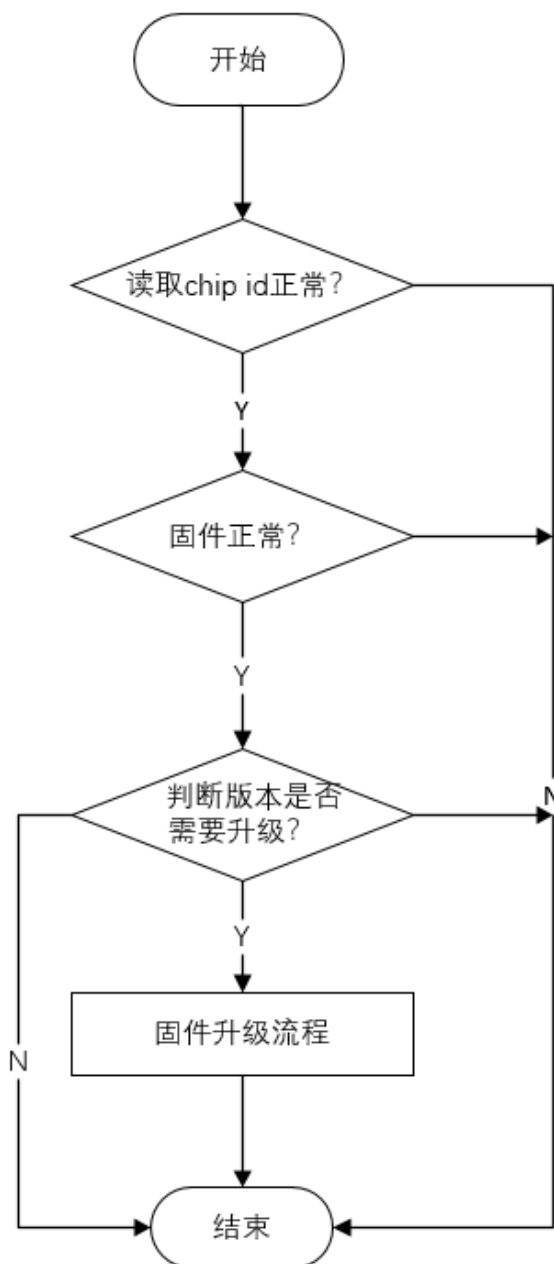
通过定时器功能, 每隔 1 秒, 执行一次 ESD 检测函数 `static void fts_esdcheck_process(void)`, 去读取固件寄存器地址 0x91 的值, 如 I2C 不通, 或者连续多次无变化, 则认为需要复位 TP, 此时 Host 需要通过 reset pin 去复位 TP。

1s 定时器中断服务入口函数根据当前平台定义, sample code 中对应的定时器处理函数为:

```
void fts_timer_interrupt_handler(void);
```

3.5 固件开机升级

3.5.1 固件开机升级流程图



3.5.2 固件开机升级移植

- 1) 把需要升级的固件 (app.i) 放入固件目录, 并且在 focaltech_upgrade_ft3169.c 中根据放入的固件目录配置升级文件的路径, 如下:

```
uint8_t fw_file_ft3169[] = {  
#include "fw_sample.i"  
};
```

- 2) 在创建的 `static void Upgrade_Task(void* parameter)` 升级任务中调用 focaltech_upgrade_ft3169.c 中自动升级函数 `int fts_fwupg_auto_upgrade(void)`, 自动升级流程与固件开机升级流程一一对应, 上电时检测到新版本即可自动执行升级。

注意: 上述升级过程中获取固件用的 app.i 格式, 是直接 include 到代码中; 如果想要使用二进制 binary 格式 app.bin, 则请根据平台读取二进制文件的方式获取 app.bin 固件的内容, 再跑升级流程。